Written Interview Responses

Education

How did you rank in your high school, in your final year in maths and hard sciences? Which was your strongest?

In my final year of high school, I ranked among the top three students in a group of about 60, both overall and in mathematics. My final-year average was 19.88/20 on Iran's grading scale. Mathematics was my strongest subject. For example, I received 20/20 in analytic geometry and algebra, 20/20 in discrete mathematics, and 19.75/20 in differential and integral calculus. I especially liked problems that required careful reasoning instead of memorization.

How did you rank in your high school, in your final year in languages and the arts? Which was your strongest?

I do not remember an official high-school ranking in language subjects, and art was not part of our high-school curriculum. English was the language subject I enjoyed most because I was also interested in the cultures it helped me understand, and my final-year English grade was 19.5/20. Later, I got 115/120 on the TOEFL iBT in 2021 and an average level of 11 on the CELPIP-General test in 2026. Outside school, I took basic music and painting classes for a few years as a child. During my bachelor's degree, I returned to music more seriously and studied the santoor (an Iranian music instrument) for two years. I still have my santoor and play it occasionally.

Please state your high school graduation results or university entrance results, along with the system used, and how to understand those.

I completed high school under Iran's 20-point grading system, and my final-year pre-university average was 19.88/20. In the 2016 Iranian University Entrance Examination, commonly called the Konkour, I ranked 17th nationwide among about 860,000 registered candidates and 12th in the mathematics and physics group of about 163,000 candidates. A lower rank is better in this system. This result gave me the option to study Computer Engineering at Sharif University of Technology, one of Iran's most selective engineering universities.

What sort of high school student were you? Outside of class, what were your interests and hobbies? What would your high school peers remember you for, if we asked them?

I was a quiet and introverted student, and I was serious about doing well in all my subjects. I think my classmates would remember me as a good student and also a good and dependable friend. Outside class, I spent extra time learning English because the standard high-school curriculum did not reach the level I wanted. I also attended swimming classes, especially during the summers. I became a good swimmer and still enjoy it very much.

Which university and degree did you choose? What other universities did you consider, and why did

you select that one?

For my undergraduate studies, I was only interested in Computer Science or Computer Engineering. Sharif University of Technology was my clear first choice because it was known as Iran's leading technical university, and its Computer Engineering program was one of the country's most selective programs. My entrance-exam rank gave me the option to choose it, and I wanted to study with strong students in a demanding environment. I did not seriously consider another university or another field. Later, I chose the M.Sc. in Computer Science at the University of Calgary because it gave me the chance to work on research software, large geospatial datasets, and applied computer science problems.

At university, did you do particularly well at any area of your degree?

During my bachelor's degree, I did particularly well in introductory and advanced programming, statistics, machine learning, and information retrieval. These courses increased my interest in building software and solving data-intensive problems. During my M.Sc., I got a 3.94/4.0 GPA and built Python/C++ systems for large Earth observation datasets. This research also resulted in a first-author publication.

Overall, what was your degree result and how did that reflect on your ability? In high school and university, what did you achieve that was exceptional?

My B.Sc. GPA was 3.69/4.0 at Sharif University of Technology, and my M.Sc. GPA was 3.94/4.0 at the University of Calgary. These results reflect my ability to learn difficult material and stay focused on long-term work. Before university, I ranked 17th nationwide among about 860,000 candidates in Iran's national entrance examination. During my M.Sc., I designed and implemented a DGGS-based system for managing spatiotemporal satellite data and published the research as first author. I was also named a Computer Science Teaching Assistant Excellence Laureate for the 2024/2025 academic year after being nominated by an instructor for my teaching work.

What leadership roles did you take on during your education?

I did not hold a formal leadership position during my education. Most of my leadership experience was technical and based on mentoring. As a teaching assistant at the University of Calgary, I guided students through database systems, big-data applications, AWS workflows, debugging, and course projects. In my graduate research, I took ownership of an open-ended problem by planning experiments, making implementation decisions, documenting and communicating the results, and turning the research idea into a working system.

Career Development

How would you describe your level of experience as a professional software engineer?

I would describe myself as an early-career software engineer who can already work independently on difficult implementation and debugging tasks. My professional experience began with backend development in 2020. Later, during my M.Sc., I worked in a hybrid industry-academic role through the BigGeo/Mitacs partnership. The company partly supported my research, but I also had software deliverables related to its needs.

My experience has therefore come from two somewhat different directions. I have built Django APIs and PostgreSQL-backed services, and I have also developed Python/C++ research software for processing, storing, and measuring large datasets. I am comfortable taking ownership of a technical problem and working through it, but I still need more hands-on experience with production cloud infrastructure and operating-system-level software.

What are your strengths as a software engineer?

My main strengths are persistence and practical problem-solving. When I accept a task, I do my best to deliver it properly, and I work through the blockers that I can solve instead of stopping at the first difficulty. I combine this with careful problem breakdown, Python/C++ implementation, debugging, performance measurement, and documentation. My M.Sc. research had many technical uncertainties, but I kept testing ideas and changing the approach until I found a direction that worked, implemented the system, completed the degree, and published the results.

What is your proudest success as a software engineer?

My proudest success is turning a difficult storage problem in my M.Sc. research into a working system for spatiotemporal Earth observation data. Normal two-dimensional wavelet methods are designed for rectangular image grids, but our DGGS used a triangular grid where every parent cell has four children. I adapted the main idea of the Haar wavelet to this structure and implemented an invertible transform that changed each group of four child values into one coarse parent value and three detail values. Applying this repeatedly created one multiresolution representation, so the system did not need to store a separate full copy of the data at every resolution.

I combined the transform with hierarchical run-length ordering and lossless TIFF/DEFLATE compression, which let us store the cells compactly and reconstruct them in a fixed order. In our published comparison, this method had a 38% compression gain for a test tensor. I also parallelized the C++ encoding pipeline and reduced the time for 10 tensors from 233.26 seconds to 26.44 seconds. This work became part of my first-author publication and is my best example of taking an unfamiliar problem through design, implementation, testing, and measurable results.

Outline the role of an engineering manager in shaping a high functioning team.

I have not managed an engineering team myself, so I think about this mostly from the perspective of someone doing the work. I work best when I understand the goal, the main constraints, and which decisions are mine to make. I also like having enough freedom to investigate a problem, with feedback early enough that I can actually use it.

To me, a manager's main job is to create those conditions for the team. They should keep priorities clear, help with blockers that an engineer cannot solve alone, and step in when a decision stays stuck. They do not need to dictate every implementation detail. I would rather have a manager who can question my approach, explain their concern, and then trust me to do the work. In a distributed team, they also need to make sure important decisions are written down, since people cannot depend on informal office conversations.

Experience

Describe your level of experience in Python, and how you have attained it.

I started using Python seriously in 2016 during my bachelor's degree. My first substantial project was a Persian text-normalization toolkit, where I wrote preprocessing rules to clean and standardize text for NLP tasks. Since then, Python has remained one of my main programming languages across almost a decade of academic, professional, and independent work.

From 2020 to 2022, I used Python and Django to develop REST APIs backed by PostgreSQL, including authentication, verification, service-plan, and payment features. During my M.Sc. and research-software work, I used Python for scientific and geospatial data pipelines with NumPy, pandas, Rasterio, GDAL, satellite-data APIs, and PyTorch. This included automating preprocessing, validating large raster datasets, and connecting Python workflows with performance-sensitive C++ code.

One substantial example was my digital elevation model super-resolution project. I built a Python and Streamlit pipeline using Rasterio and the Google Earth Engine API to download and preprocess elevation tiles from mountainous regions around the world. I then used PyTorch to train and evaluate deep residual networks for four-times super-resolution. The evaluation included standard error measures as well as terrain-aware measures such as slope, TRI, TPI, and wavelet-based errors. Overall, I am comfortable using Python for backend services, data processing, scientific software, and machine-learning workflows.

When did you start working on Linux? Describe your level of experience as a user and developer on Linux.

I started using Ubuntu regularly in 2016, when I entered university and got my first personal laptop. I chose Ubuntu as its main operating system, and it has remained the main environment where I study, develop software, and do research. I now have almost a decade of continuous experience with Ubuntu as both a daily user and a developer.

On Ubuntu, I regularly use the command line, Python environments, Git, scripting, data-processing tools, and Python/C++ development workflows. I have managed software with `apt`, used SSH to connect to remote systems, managed file permissions, worked with services and scheduled tasks, and configured Docker development environments. I am comfortable diagnosing dependency and environment problems and working without a graphical interface when needed. I am an experienced Ubuntu user and application developer, although I am not a kernel or distribution-maintenance specialist.

How do you address software performance, systematically, in your products and in your software engineering practices?

I start by measuring before I optimize. I choose a representative workload and baseline, measure the main stages, and try to determine whether the limit comes from I/O, the algorithm, memory, or CPU work. I have used C++ profiling, Python timers, and small benchmark scripts to find expensive operations. I then change one relevant factor at a time, repeat the measurement under similar conditions, and check that the result is still correct.

I used this process in my satellite-data encoding pipeline. Measuring each stage showed where the time was being spent and which work could run independently. I parallelized tensor encoding in C++ and compared the same 10-tensor workload with one and 10 processing units. The runtime dropped from

233.26 seconds to 26.44 seconds. I recorded the workload and hardware configuration so that the result could be reproduced.

How do you prefer to drive documentation for your products?

I prefer to write documentation while I develop the software instead of trying to reconstruct everything at the end. Useful documentation should help someone run, test, understand, or change the system. I value clear setup instructions, examples, API behavior, and short explanations of decisions that are not obvious from the code.

During my work with BigGeo, I prepared weekly reports about progress, results, blockers, and next steps. I also wrote documentation for APIs I developed. For performance and storage experiments, I kept the configurations and results in structured Google Sheets so that I could compare experiments and keep the numbers in context. My research paper was the more formal documentation of the final system, its algorithms, evaluation, and limitations. These documents served different purposes, but together they made the work easier to follow and reproduce.

Describe your experience with QEMU/KVM, LXC/LXD and cloud computing.

My direct virtualization experience has been with VMware for running local virtual machines. I have not yet worked directly with QEMU/KVM or LXC/LXD. My related experience includes using Ubuntu as my main operating system, managing development environments, and working with Docker containers. This gives me a useful foundation, but I would need to build hands-on experience with QEMU/KVM and LXD.

My cloud experience comes mainly from projects and teaching rather than production operations. I have personally used, and helped students use, AWS services in a Developing Big Data Applications course. I used EC2 and S3 most, with some experience in Lambda, EBS, and DynamoDB. I also helped students understand and debug cloud-based data-processing and storage workflows. I would like to extend this foundation into Ubuntu cloud images and provisioning.

How would you describe your experience in Linux system fundamentals such as networking, storage, and security?

I have practical knowledge of Linux fundamentals from using Ubuntu as my main development environment since 2016, although I am not a systems specialist. For networking, I have regularly used and troubleshot SSH connections and ports, and I have configured Docker port mappings for containerized services. My backend work also required working with HTTP APIs and database connections.

For storage, I have worked with Linux filesystems, permissions, local and remote datasets, Docker storage, PostgreSQL, S3, and EBS. Storage was also a main part of my research, where I designed compact representations for large satellite datasets and measured storage and retrieval tradeoffs. My security experience includes Linux permissions and backend authentication features. I still want to build deeper knowledge of network and operating-system security.

How do you think about software quality?

For me, software quality starts with whether the program gives the correct result for real inputs, including the awkward cases. It should also fail in a way that can be understood. A program is difficult to trust if it

produces a wrong result silently, or if nobody can tell what happened when it fails.

In my own work, I use automated tests such as `pytest`, some assertions, and a lot of logging while debugging. I try to keep inputs reproducible and compare results with a known output or baseline when possible. Research software taught me another side of quality: code can run exactly as written and still support a wrong conclusion if the data split or evaluation method is weak. Because of that, I also check how a result was produced, not only whether the program completed successfully.

Describe a case where it was very difficult to test code you were writing, but you found a reliable way to do it.

In my DEM super-resolution project, testing was difficult because terrain data are spatially related. With a random tile split, neighboring or very similar terrain could appear in both the training and test sets. The model could then produce strong numbers without showing that it could generalize to a truly unseen region.

I handled this by separating the data geographically and holding out regions from training, so the test tiles came from locations the model had not seen. I also used more than one pixel-error measure. Along with RMSE, I used slope, terrain ruggedness index (TRI), topographic position index (TPI), and wavelet-based errors. The geographic split reduced data leakage, while the other measures checked whether the reconstructed elevation kept important terrain structure. The results were more conservative, but I trusted them more than the results from a random split.

What would you like to achieve in career development and skills development?

My immediate goal is simple: I want to become a senior software engineer. I know that title should come from experience, not only from learning more tools. I want to own larger pieces of real systems, see the longer-term results of my technical decisions, and learn from both successful releases and mistakes. Linux and cloud infrastructure are the areas where I most want to deepen my experience, while continuing to improve in Python, testing, and performance work.

Later, I would like to mentor other engineers and possibly take on technical leadership. Engineering management may also interest me after I have enough experience to be genuinely useful to other people. I do not want to rush toward that before becoming a strong senior individual contributor myself.

Context

How are you involved in open source software?

So far, I have been more of a long-term open-source user and developer than an upstream contributor. Ubuntu has been my main operating system since 2016, and my backend and research work has depended heavily on projects such as Python, Django, PostgreSQL, Docker, PyTorch, GDAL, and Rasterio. I have combined these tools into working systems and used their documentation and communities to investigate integration problems. I have not yet contributed to a third-party repository, but becoming a responsible upstream contributor is an important next step for me.

Describe any significant contributions to open source, with links where possible.

I have not yet made a significant open-source contribution or published a public repository that I would present as one, so I do not have a contribution link to provide. Most of my software work has happened in professional, academic, or private projects while making extensive use of open-source tools. I want to learn the upstream workflow through useful and reviewable contributions, but I do not want to present my existing work as something it is not.

What do you think are the key ingredients of a successful open source project?

When I depend on an open-source project, I notice quickly whether I can trust its behavior and documentation. A successful project should solve a real problem and let people use it without first learning every internal detail. It also needs maintainers who keep the project moving, review contributions seriously, and avoid making each release a surprise for existing users.

Rasterio is the clearest example from my own work. Raster data have many awkward details, but Rasterio gave me a Python interface that handled most of the low-level work in a consistent way. Its documentation and examples helped me solve real problems without rebuilding raster operations myself. That combination is what made it valuable to me, not simply the number of features it had.

Why do you most want to work for Canonical?

What attracts me most is the combination of Python, Linux, and cloud engineering. Python has been one of my main languages for almost a decade, and Ubuntu has been my main operating system since I entered university. Cloud infrastructure is where I most want to build deeper experience, so this role connects what I already do well with what I want to learn next.

The scale also matters to me. Improvements to Ubuntu cloud images can affect many real systems across different providers and environments, so correctness, performance, and edge cases are important. I also want to move from mainly using open-source software to contributing within a mature open-source engineering process. Canonical is one of the few places where these interests come together in the same role.

Which other companies are building the sort of products you would like to work on?

Microsoft is the first company that comes to mind because it builds many kinds of engineering products: operating systems, Azure, developer platforms such as GitHub and Visual Studio Code, databases, and AI tools. I like companies that build foundational products which other developers use to create their own systems, and Microsoft does this in several areas.

AWS is another example because of the depth of its cloud infrastructure. I have used EC2, S3, Lambda, EBS, and DynamoDB, and this made me curious about how these services are built and operated reliably at scale. I would be interested in work involving developer infrastructure, distributed systems, data platforms, or software supporting cloud services. Microsoft and AWS are not the same as Canonical, but all three build foundational technology used by many other systems.

What do you think Canonical needs to improve in its engineering and products?

I do not have enough direct experience with Canonical's internal engineering to claim a specific technical

weakness. From my perspective as a long-term Ubuntu user who has not yet contributed upstream, Canonical could keep improving the path from experienced user to first-time contributor. Clear architecture overviews, reproducible setup and test instructions, and realistic examples of first contributions could help more users become long-term contributors.

What do you think is the biggest opportunity for Canonical in this arena?

To me, the biggest opportunity is where cloud computing and AI/ML meet. My own machine-learning work was at research scale, but even there I saw how quickly compute, storage, and reproducibility become harder as a project grows. Larger AI systems depend even more on reliable cloud infrastructure. Ubuntu is already common in both areas, so Canonical has a good chance to make AI workflows easier to reproduce and move between different clouds.

Who do you think are key competitors to Canonical? How do you think Canonical should plan to win that race?

It depends on which part of Canonical's business we are considering. Red Hat and SUSE are more direct competitors in enterprise Linux, while Microsoft and AWS compete with Canonical in parts of the cloud market. I do not think Canonical should try to copy all of them. Ubuntu is already familiar to many developers, and that familiarity is a real advantage. If moving from a personal machine or a small research project to a large cloud deployment creates as little friction as possible, I think that is a strong way for Canonical to compete.